

Here it is as plain text, ready to paste directly into markdown cells (no code fencing):

For the Objective 5 discussion cell:

Observations: Rewards, Termination, and Environment Behaviour

Running the random agent for 50 episodes on each environment revealed clear differences in how reward and termination behave:

CartPole-v1 — Average reward 21.82 (min 9, max 51), with all 50 episodes ending via `terminated=True`. Since the reward is +1 for every timestep the pole stays upright, total reward is just the number of steps survived. A random agent has no strategy to counteract the pole's tilt, so it falls quickly and consistently — well short of the 500-step cap — meaning the environment's own failure condition (not the time limit) always ends the episode.

MountainCar-v0 — Average reward exactly -200.00 on every single episode, with all 50 episodes `truncated=True` and zero `terminated`. The car needs directed back-and-forth momentum to escape the valley, which random actions essentially never produce within 200 steps. Because the reward is -1 per timestep with no partial credit, every failed episode racks up the exact same penalty — this is why the result is completely flat with zero variance, unlike CartPole.

FrozenLake-v1 — Average reward 0.00, all 50 episodes `terminated=True`. Reward is sparse (0 for every step, +1 only at the goal), and the "slippery" stochastic transitions mean an action doesn't reliably move the agent where intended. A random agent falls into a hole almost immediately in every run, so it never earns the +1 goal reward and never runs long enough to hit the 100-step limit.

Taxi-v4 — Average reward -745.44 (min -938, max -98), with only 6/50 episodes `terminated=True` and 44/50 `truncated=True`. Taxi combines a large discrete state space (500 states) with penalty-heavy rewards (-1 per step, -10 for illegal pickup/drop-off, +20 for success). A random agent racks up frequent illegal-action penalties and rarely completes a full pickup-and-dropoff sequence within 200 steps, so most episodes simply run out of time instead of succeeding.

Pattern across all four: `terminated` means the environment's own rules (success or failure) ended the episode; `truncated` means the step limit was hit first. Environments with quick, easily triggered failure conditions (CartPole, FrozenLake) almost always terminate. Environments that require sustained, coordinated behaviour to succeed (MountainCar, Taxi) almost always truncate under a random policy, since reaching the goal by chance within the step budget is highly unlikely.

For the conclusion cell:

Conclusion

This lab explored the Gymnasium API across four benchmark environments spanning different structural properties: continuous vs. discrete observation spaces, dense vs. sparse reward, and deterministic vs. stochastic transitions.

Two environments (CartPole-v1, MountainCar-v0) use continuous Box observation spaces suited to physics-based control, while the other two (FrozenLake-v1, Taxi-v4) use discrete, finite state spaces that map naturally onto tabular RL methods — Q-tables, Dynamic Programming, and Monte Carlo methods, which this course covers next.

The random-agent baseline made the practical cost of "no learning" concrete: CartPole survived only ~22 steps on average out of a possible 500, MountainCar and FrozenLake never once succeeded across 50 episodes each, and Taxi accumulated heavy negative reward from illegal actions. These results give a clear, numeric baseline to compare against once tabular algorithms (introduced later in the course) are implemented on the same environments — any real learning algorithm should substantially outperform these random-agent numbers.

GitHub Link of whole assignment: https://github.com/leekhithnunna/RL_Labs.git